L1-coverage Exploration Problem in Tightrope Environment

Lufan Cao
Qiuzhen College
Tsinghua University
caolf23@mails.tsinghua.edu.cn

1 Standard Method Review

Our reproduction is based on the CODEX.W method proposed in *Scalable Online Exploration via Coverability* [Amortila et al., 2024], mainly Algorithm 4 and Algorithm 5. We also refer to Algorithm 7, namely Policy Search by Dynamic Programming (PSDP)[Bagnell et al., 2003], as the policy optimization subroutine.

1.1 Overview of CODEX.W

CODEX.W is a model-free exploration algorithm motivated by the L_1-coverage objective. Its goal is to construct a sequence of policy covers

$$p_1, p_2, \ldots, p_H,$$

where each p_h is a mixture distribution over policies that can cover the state space at layer h.

The algorithm proceeds layer by layer. For each layer h, it learns T policies

$$\pi_{h,1}, \pi_{h,2}, \ldots, \pi_{h,T},$$

and defines the policy cover at this layer as the uniform mixture

$$p_h = \mathrm{Unif}\big(\pi_{h,1} \circ_h \pi_{\mathrm{unif}}, \ldots, \pi_{h,T} \circ_h \pi_{\mathrm{unif}}\big).$$

During exploration, one policy is sampled uniformly from this cover and then executed in the environment.

The main idea is that, when learning the t-th policy at layer h, the algorithm tries to find a policy that reaches states not already well covered by the previously learned policies

$$\pi_{h,1}, \ldots, \pi_{h,t-1}.$$

To achieve this, CODEX.W constructs an intrinsic reward using weight function estimation. This reward is large on transitions that are likely under the true dynamics but are not sufficiently covered by the current policy cover. The resulting reward is then optimized by a policy optimization subroutine.

1.2 Weight Estimation Method

For a fixed layer h and iteration t, CODEX.W estimates the weight function

$$w_h^t(x_h \mid x_{h-1}, a_{h-1}) = \frac{P_{h-1}^{M^\star}(x_h \mid x_{h-1}, a_{h-1})}{\sum_{i<t} d_h^{M^\star, \pi_{h,i}}(x_h) + P_{h-1}^{M^\star}(x_h \mid x_{h-1}, a_{h-1})}.$$

Intuitively, this quantity is large when the transition (x_{h-1}, a_{h-1}, x_h) is feasible under the environment dynamics but the next state x_h has not been frequently reached by the previous policies.

Preprint.

To estimate this weight function, the algorithm defines a sampling distribution

$$q = \frac{1}{2} p_{h-1} + \frac{1}{2(t-1)} \sum_{i<t} \pi_{h,i} \circ_{h-1} \pi_{\text{unif}}.$$

The first term uses the previous-layer policy cover, while the second term uses the policies already learned at the current layer. This mixture ensures that the samples are collected from both the existing cover and the newly constructed policies.

The algorithm then constructs two datasets D_1 and D_2. First, it samples trajectories using policies drawn from q and obtains transition tuples

$$(x_{h-1}, a_{h-1}, x_h).$$

These tuples are added to D_1. To form D_2, the algorithm pairs the same preceding state-action pair (x_{h-1}, a_{h-1}) with an alternative next state $\widetilde{x}_h$ sampled from the distribution induced by the previous policies. Thus, D_1 represents real transitions under the environment, while D_2 represents contrastive transitions whose next states are drawn from the already-covered distribution.

The weight function is estimated by solving the following density-ratio estimation objective:

$$\widehat{w} = \arg\max_{w \in \mathcal{W}_h} \left\{ \widehat{\mathbb{E}}_{D_1} \left[\log w(x_h \mid x_{h-1}, a_{h-1}) \right] - t \cdot \widehat{\mathbb{E}}_{D_2} \left[w(x_h \mid x_{h-1}, a_{h-1}) \right] \right\}.$$

This objective encourages w to assign large values to transitions in D_1, while penalizing transitions in D_2. Therefore, the learned weight function can be interpreted as a contrastive reward that favors newly reachable or insufficiently covered states.

After estimating $\widehat{w}$, CODEX.W defines the reward for the policy optimization step as

$$r_{h,t}^{h-1}(x_{h-1}, a_{h-1}, x_h) = \widehat{w}(x_h \mid x_{h-1}, a_{h-1}),$$

and sets the rewards at all other layers to zero:

$$r_{h,t}^{h'} = 0, \qquad h' \neq h-1.$$

Thus, the policy optimization subroutine only needs to optimize a single-step intrinsic reward at layer $h-1$.

1.3 Policy Optimization Method

After constructing the intrinsic reward, CODEX.W calls a policy optimization subroutine to compute

$$\pi_{h,t} \approx \arg\max_{\pi \in \Pi} \mathbb{E}_{M^\star, \pi} \left[\widehat{w}(x_h \mid x_{h-1}, a_{h-1}) \right].$$

In the paper, this subroutine can be any method that approximately solves the above reward-driven policy optimization problem. The theoretical analysis uses Policy Search by Dynamic Programming (PSDP).

PSDP works backward from the target layer to the initial layer. Given policy covers $p_1, \ldots, p_{h-1}$, it collects exploratory data from these covers and performs approximate dynamic programming using a value function class $\mathcal{Q}$. At each step, PSDP estimates the corresponding Q-function by future rollout and then chooses the action that maximizes the estimated value. In this way, PSDP can utilize the past exploration covers and is efficent is this case.

2 Our Improved Methods

2.1 Direct Computation of the Weight Function

Since our experiments are conducted in a tabular environment, the weight function can be computed directly instead of being optimized by gradient ascent over a neural function approximator.

In the original method, the weight estimation step solves an objective of the form

$$\max_{w} \ \widehat{\mathbb{E}}_{D_1}[\log w] - t \cdot \widehat{\mathbb{E}}_{D_2}[w].$$

For a tabular state-action-next-state tuple (s, a, s'), let $n_1(s, a, s')$ denote the number of occurrences of this transition in D_1, and let $n_2(s, a, s')$ denote the number of occurrences in D_2. Then the optimization reduces to the one-dimensional problem

$$\max_{w>0} \; n_1 \log w - t\, n_2\, w.$$

This objective has a closed-form optimum obtained by setting the derivative to zero:

$$\frac{n_1}{w} - t n_2 = 0 \quad \Longrightarrow \quad w^\star = \frac{n_1}{t n_2},$$

with appropriate smoothing when $n_2 = 0$.

Therefore, in the tabular setting, we can compute the reward exactly and efficiently without stochastic optimization. This significantly reduces variance, improves numerical stability, and accelerates training.

2.2 Reward at Every Step Instead of Only the Final Step

Our main motivation is that the original method aims to make the policy cover uniformly explore the state space at a specific layer h. However, in practical applications, the number of policies in the cover must remain small due to computational constraints. For example, we may only use $T = 10$ policies, while the number of reachable states can be much larger.

Moreover, each policy is trained by reinforcement learning and may converge to a nearly identical optimal solution. As a result, if only the final step is rewarded, then at most T terminal positions may be covered, which is far from sufficient for large environments.

The theoretical framework of the original paper assumes sufficiently large T, but this assumption is often unrealistic in practice.

To address this limitation, we modify the objective. Instead of covering the state space only at step h, we encourage coverage over the entire trajectory. That is, during a rollout

$$s_1 \to s_2 \to \cdots \to s_H,$$

we reward the agent whenever it visits under-explored states at any time step.

Accordingly:

- rewards are assigned at every step rather than only at step $h - 1$;
- weight estimation uses samples collected from all layers, starting from the first step;
- policy optimization maximizes trajectory-wide exploration.

This transforms the problem from *terminal-state coverage* into *path-wise coverage*, which is substantially easier in long-horizon environments. As the horizon increases, the number of states visited along a trajectory can grow much faster than the number of terminal states alone.

2.3 State Reward Instead of Transition Reward

We further observe that many transitions stored in D_2 are inefficient or weakly informative. In the original transition-based construction, a state-action pair (s_{h-1}, a_{h-1}) may be matched with a next state $\widetilde{s}_h$ that is not realistically reachable from that pair. Such samples contribute noise and reduce data efficiency.

More importantly, our true objective is not to reward a specific transition, but rather to encourage the agent to reach rarely visited states. In many exploration tasks, *how* the agent reaches a state is less important than *whether* the state is reached.

Therefore, we introduce a state-based reward design.

Dataset construction.

- In D_1, we still store full transitions (s, a, s').
- In D_2, we only store visited next states s'.

Reward definition. For each state s', we define

$$w(s') = \frac{1}{|\mathcal{N}(s')|} \sum_{(s,a):\,(s,a,s') \text{ valid}} \arg\max_{w} \left[\#_{D_1}(s,a,s') \cdot \log w - t \cdot \#_{D_2}(s') \cdot w \right],$$

where $\#_{D_1}$ and $\#_{D_2}$ denote counts in the two datasets, and $\mathcal{N}(s')$ is the set of predecessor state-action pairs leading to s'.

During policy optimization, the reward at each step is simply the value of the visited next state:

$$r(s,a,s') = w(s').$$

This design has several advantages:

- higher sample efficiency;
- fewer reward parameters than transition-based modeling;
- simpler optimization and faster convergence.

2.4 Tabular Policy and Neural Network Approximation

For the policy optimization stage, we implemented two different designs for the Q-function.

Tabular policy. The first design uses a tabular Q-table. This approach is simple and highly effective in grid-world tasks, since the state space is discrete and relatively small. It allows exact updates, stable learning dynamics, and straightforward debugging.

However, tabular methods do not scale well to large or continuous state spaces.

Neural network approximation. To improve scalability, we also design a neural-network-based Q function. The network takes spatial coordinates (x, y) as input, and we apply sinusoidal positional encoding before feeding them into the model. This encoding helps the network represent periodicity, locality, and fine-grained spatial structure, which is especially useful for long grid-like environments such as narrow corridors or large maps. Compared with raw coordinate inputs, positional encoding typically improves generalization to unseen locations and stabilizes optimization.

Policy extraction. Even in continuous or large-scale settings, we do not need to train a separate policy network. Instead, actions can be extracted directly from the learned Q-function using

$$\pi(s) = \arg\max_{a} Q(s,a),$$

or an ϵ-greedy variant during exploration.

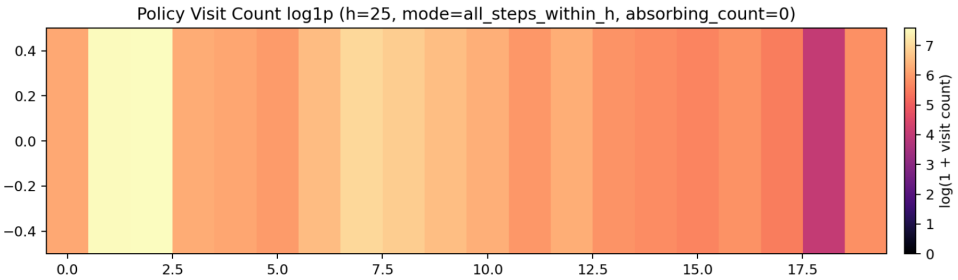
This keeps the architecture simple while preserving the flexibility of function approximation.

3 Experiment Results

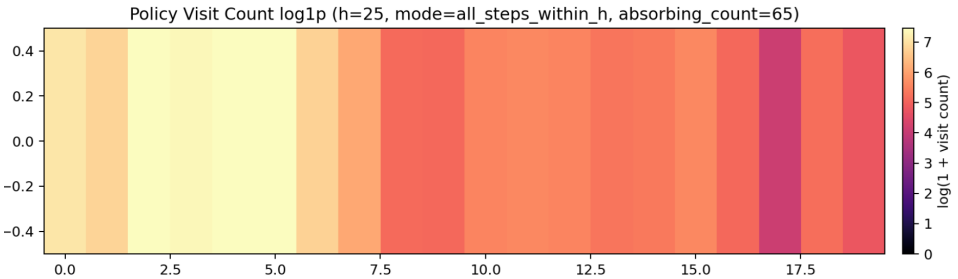
We evaluate our method on three grid-world environments of increasing size in order to study both exploration quality and scalability. For all figures, brighter colors indicate states visited more frequently, and all heatmaps are shown on a $\log(1+x)$ scale for better visualization.

Environment 1: 1×20. We first consider a one-dimensional corridor of size 1×20. Within fewer than 25 layers, both the tabular Q implementation and the neural-network-based implementation successfully cover all reachable states.

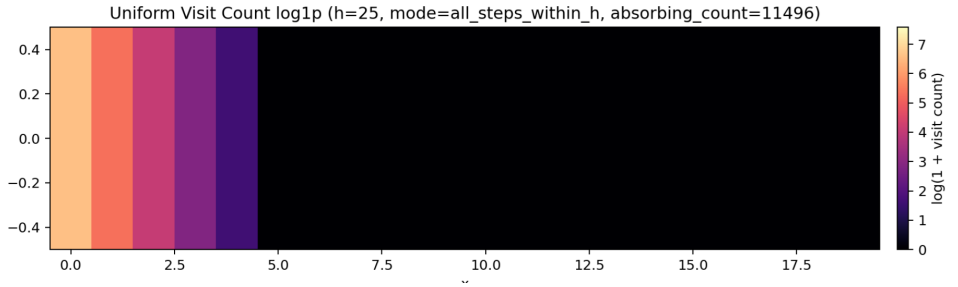
However, the tabular method produces a visibly more uniform visitation distribution. A likely explanation is that while neural networks provide better generalization, optimization in small environments can be less stable and may introduce unnecessary approximation noise. For reference, we also include the uniform random policy baseline, which explores the environment but exhibits weaker directed coverage.



(a) Tabular Q method on 1×20.



(b) Neural-network method on 1×20.



(c) Uniform random policy baseline.

Figure 1: State visitation heatmaps on the 1×20 environment. Both learned methods achieve full coverage, while the tabular version is more uniform than the neural version. The uniform random baseline explores the corridor but lacks the same directed balancing effect.

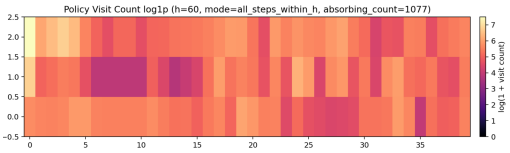
Environment 2: 3×40**.** We next test a larger two-dimensional environment of size 3×40. The overall trend is similar to the previous setting. Both methods cover almost all states within 60 layers, and the tabular version again shows a slightly more uniform state distribution.

Interestingly, the neural-network version reaches full coverage much earlier: it covers all states by layer 46, whereas the tabular version requires approximately 53 layers. This suggests that function approximation and spatial generalization help accelerate exploration in larger environments. We again compare against a uniform random policy, which spreads locally but cannot efficiently propagate to distant states.

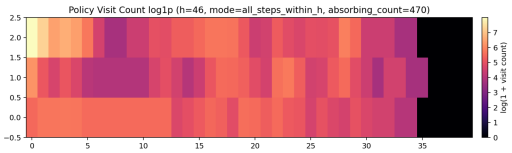
Environment 3: 5×100**.** Finally, we consider a substantially larger environment of size 5×100. Training in this setting is significantly more expensive, so experiments are still in progress.

For the case $T = 10$, the neural-network version has currently been trained up to layer $h = 65$. The corresponding comparison between tabular, neural, and uniform-policy baselines is shown below.

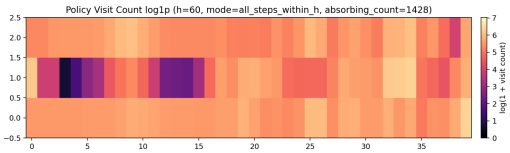
The results indicate that the neural-network version benefits substantially from generalization in large environments and achieves higher coverage more quickly. Moreover, both learned methods continue to improve steadily as training proceeds.



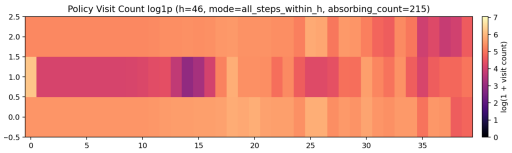
(a) Tabular Q method, $h = 60$.



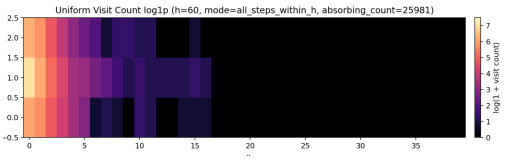
(b) Tabular Q method, $h = 46$.



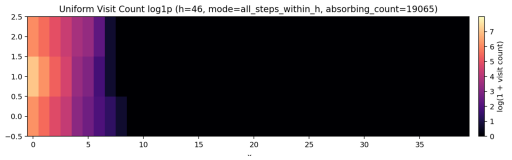
(c) Neural network method, $h = 60$.



(d) Neural network method, $h = 46$.

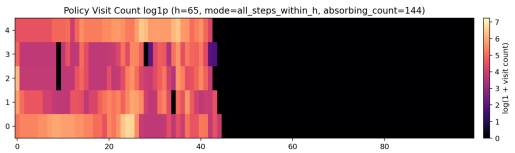


(e) Uniform random policy baseline, $h = 60$.

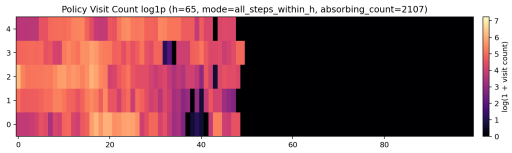


(f) Uniform random policy baseline, $h = 46$.

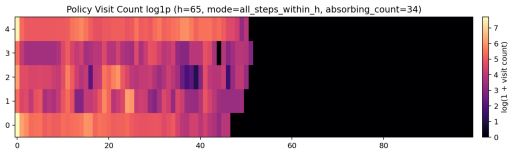
Figure 2: Coverage heatmaps on the 3×40 environment. Top row shows later-stage training results ($h = 60$), while bottom row shows earlier-stage results ($h = 46$). The neural method reaches full coverage earlier, whereas the tabular version remains more uniform. Both substantially outperform the uniform random baseline in long-range coverage efficiency.



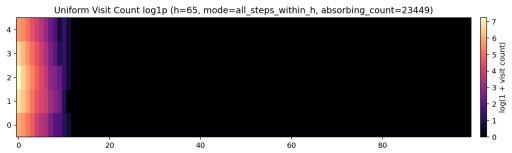
(a) Tabular Q method, $T = 10$, $h = 65$.



(b) Neural network method, $T = 10$, $h = 65$.



(c) Tabular Q method, $T = 20$, $h = 65$.



(d) Uniform random policy baseline.

Figure 3: Coverage heatmaps on the 5×100 environment. Large-scale exploration remains challenging. The neural version expands coverage faster than the tabular version under the same cover size ($T = 10$). Increasing the number of policies to $T = 20$ substantially improves uniformity. The uniform baseline explores only a limited local region and fails to reach broad coverage efficiently.

We also observe non-uniform regions and uncovered holes in both methods when $T = 10$. This is expected, since using only ten policies is insufficient to uniformly cover a state space of size 5×100.

To investigate this issue, we additionally test the setting $T = 20$. Even with only the tabular version currently available at layer 65, the coverage becomes noticeably more uniform. Although the neural-network version has not yet reached the same training horizon, we expect a similar or stronger improvement once training is completed.

Summary. Across all three environments, our results demonstrate a consistent trade-off:

- Tabular methods provide more stable and uniform coverage in small environments.
- Neural methods generalize better and scale more effectively in larger environments.
- Increasing the number of policies in the mixture cover significantly improves exploration quality in large maps.

These findings support the practical value of combining coverage-driven exploration with function approximation.

4 Ablation Study

We conduct an ablation study based on the tabular implementation in order to evaluate the contribution of our two main modifications:

- Reward at Every Step Instead of Only the Final Step;
- State Reward Instead of Transition Reward.

All experiments in this section are performed using the tabular method, so that the effect of each design choice can be isolated without the additional influence of function approximation.

Ablation 1: Reward at Every Step. We first evaluate the effect of assigning rewards at every step rather than only at the final target layer. Experiments are conducted on the 3×40 environment with $T = 10$, trained for 60 layers.

Without the *Reward at Every Step* modification, the original method theoretically aims to make the final positions uniformly cover the entire state space. However, since only $T = 10$ policies are used in the mixture, the algorithm can learn only around ten dominant terminal positions. Consequently, the terminal-state distribution cannot cover the full environment.

This limitation also propagates backward to earlier steps, leading to poor coverage across the whole trajectory.

In contrast, our modified method achieves substantially higher coverage at every step. At the same time, the final-step distribution still retains the expected structure of scattering dominant terminal locations, corresponding to the ten policies in the cover. (Because there's a uniform step after our exploration policy, so the final-step positions count is greater than ten.)

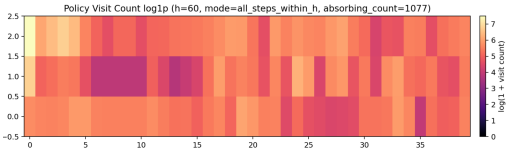
These results confirm that optimizing only terminal states is inadequate when the number of policies is small, whereas rewarding the entire trajectory makes exploration substantially more effective.

Ablation 2: State Reward vs. Transition Reward. We next evaluate the effect of replacing transition-based rewards with state-based rewards.

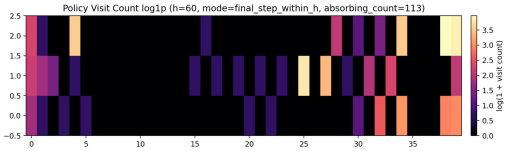
Our experiments show that using transition rewards can still gradually expand exploration and eventually cover the environment with a fairly uniform distribution. Therefore, the original transition-level reward is indeed meaningful.

However, compared with our state-based reward design, training with transition rewards is noticeably slower. This is consistent with our analysis: transition rewards involve a larger reward space and contain many less informative samples, whereas state rewards focus directly on the occupancy of target states.

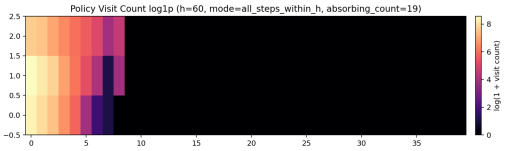
We compare both methods on the 3×40 environment after training for 60 layers.



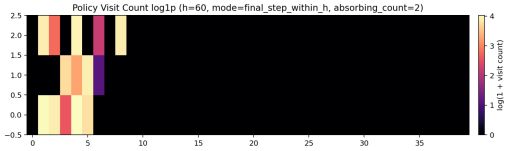
(a) Reward at every steps (ours), every-step coverage.



(b) Reward at every steps (ours), final distribution.

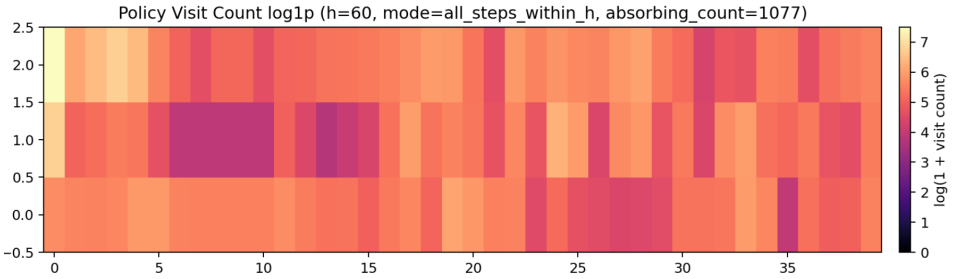


(c) Reward at final step (original), every-step coverage.

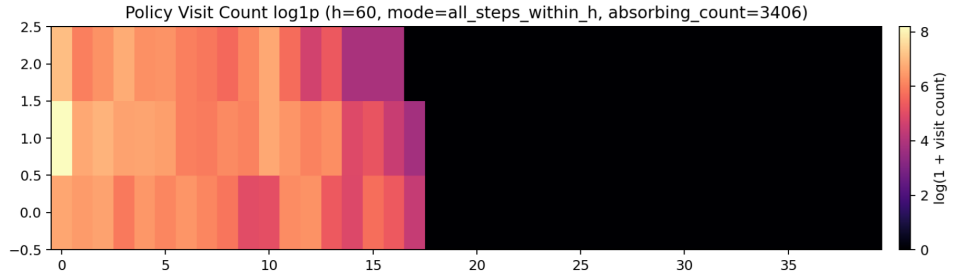


(d) Reward at final step (original), fina distribution.

Figure 4: Ablation study for Reward at Every Step on the 3×40 environment ($T = 10$, 60 layers). Our method significantly improves coverage over the full trajectory.



(a) State-based reward (ours).



(b) Transition-based reward (original).

Figure 5: Ablation study for State Reward vs. Transition Reward on the 3×40 environment (60 layers). Both methods explore successfully, but the state-based reward converges faster.

Discussion. The ablation results support both of our proposed modifications:

- Rewarding every step greatly improves global trajectory coverage when the policy cover size T is limited.
- State-based rewards are more data-efficient and lead to faster optimization than transition-based rewards.

Together, these findings explain why our improved method performs better than the original formulation in practical large-scale environments.

5 Discussion, Limitations, and Future Work

5.1 Discussion

Our empirical results provide several useful insights regarding coverage-driven exploration in practical environments.

First, although the original CODEX.W framework is theoretically well-motivated, its practical performance can degrade when the number of policies in the cover is small. In theory, increasing the cover size T allows the policy mixture to approximate broad state-space coverage. However, in realistic experiments, using a large T is often computationally expensive. When T is small (e.g., $T = 10$), the final layer can only contain a limited number of dominant terminal positions, which restricts exploration quality in large environments. So our experiments suggest that optimizing only the final-step coverage objective is insufficient. By contrast, assigning intrinsic rewards throughout the entire trajectory significantly improves overall visitation coverage. This indicates that trajectory-level exploration objectives may be more effective than purely terminal-state objectives in practice.

Second, the comparison between tabular and neural implementations reveals a clear trade-off. Tabular methods are more stable and often produce more uniform visitation patterns in small discrete environments. Neural networks, however, provide stronger generalization and scale much better to larger environments. In particular, on larger maps, the neural method often reaches new states earlier than the tabular baseline.

Finally, our state-based reward design appears more efficient than the original transition-based reward. Since exploration fundamentally aims to reach new states rather than optimize specific transitions, directly rewarding under-visited states leads to faster convergence and simpler optimization.

Overall, these findings suggest that practical exploration performance depends not only on theoretical coverage guarantees, but also on reward design, policy parameterization, and computational constraints.

5.2 Limitations

Despite promising empirical performance, our current study still has several limitations.

First, some large-scale experiments are not yet fully converged. In particular, the 5×100 environment requires substantially more training time, so our current results should be viewed as intermediate rather than final outcomes.

Second, the neural-network implementation introduces additional hyperparameter sensitivity. Learning rate, exploration noise, reward normalization, and network architecture can all influence performance. More systematic tuning may further improve results.

Third, all experiments in this work are conducted on structured grid-world environments. Although these environments are useful for analyzing exploration behavior, they remain simpler than many realistic reinforcement learning tasks involving high-dimensional observations or continuous actions.

Fourth, while our modifications perform well empirically, we have not yet established corresponding theoretical guarantees. In particular, the every-step reward formulation and the state-based reward formulation are currently motivated by practice rather than formal coverability analysis.

Finally, our current implementation still relies on tabular counting or explicit occupancy statistics in several components, which may become difficult to maintain in very large or continuous state spaces.

5.3 Future Work

A natural next direction is to develop more scalable optimization pipelines that combine offline and online reinforcement learning.

In our setting, standard on-policy methods such as PPO [Schulman et al., 2017] cannot be applied directly in a naive manner, because successful optimization depends on the exploratory data generated by previously learned policy covers. Without this exploration foundation, purely on-policy learning may fail to discover informative states efficiently.

Therefore, a promising direction is to combine offline RL methods with online fine-tuning. For example, methods such as Implicit Q-Learning (IQL) [Kostrikov et al., 2021] can first leverage the replay data collected from previous covers, and then be combined with online interaction to continue improving the policy. Such hybrid pipelines may be substantially more sample-efficient than purely online optimization.

A second direction is to replace the current tabular weight estimation procedure with neural function approximation. Instead of explicitly counting occurrences in D_1 and D_2, one may train a neural network to predict weights or approximate density ratios directly from state transitions. This would allow the method to scale naturally to continuous states, image observations, and large environments.

A third direction is to evaluate the method in more challenging domains. Examples include Atari environments with pixel observations, sparse-reward navigation tasks, and embodied control problems in robotics. These settings would provide a stronger test of whether coverage-driven exploration remains effective beyond grid-world benchmarks.

Finally, it would be valuable to develop new theory explaining our practical modifications. In particular, establishing guarantees for trajectory-wide rewards, state-based intrinsic rewards, and neural weight estimation would help bridge the gap between elegant theoretical frameworks and practical exploration algorithms.

References

Philip Amortila, Dylan J. Foster, and Akshay Krishnamurthy. Scalable online exploration via coverability, 2024. URL `https://arxiv.org/abs/2403.06571`.

J. Bagnell, Sham M Kakade, Jeff Schneider, and Andrew Ng. Policy search by dynamic programming. In S. Thrun, L. Saul, and B. Schölkopf, editors, *Advances in Neural Information Processing Systems*, volume 16. MIT Press, 2003. URL `https://proceedings.neurips.cc/paper_files/paper/2003/file/3837a451cd0abc5ce4069304c5442c87-Paper.pdf`.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning, 2021. URL `https://arxiv.org/abs/2110.06169`.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL `https://arxiv.org/abs/1707.06347`.

A Experiment Details

A.1 Codebases and Variants

We report experiments from two closely related implementations:

- `codex_state_w`: tabular PSDP policy optimization (empirical mean regression for $Q(s, a)$).
- `codex_state_w_nn`: neural PSDP policy optimization, where a small MLP is trained per timestep and then evaluated on visited states to produce tabular action values.

Both variants share the same environment, weight-estimation routine, rollout logging, and evaluation pipeline; they differ only in the policy optimization module.

A.2 Environment

We use `TightropeEnv`, a deterministic grid corridor with absorbing failure:

- State: (x, y) with $x \in \{0, \ldots, \texttt{length}-1\}$ and $y \in \{0, \ldots, \texttt{width}-1\}$, plus one absorbing state $(\texttt{length}, \texttt{width})$.
- Actions: $\{0, 1, 2, 3\}$ for up/down/left/right.
- Transition: leaving the corridor in the y direction enters the absorbing state; the absorbing state self-loops.
- Reward: 1 at the far-right goal column ($x = \texttt{length} - 1$), else 0.
- Episodes do not terminate on goal or lava by `terminated`; instead, they continue until `max_steps` by `truncated`. This preserves fixed-horizon rollouts required by layered coverage construction.

A.3 Algorithmic Pipeline

Both codebases implement the same high-level CODEX.W loop:

1. Initialize p_1 as uniform random policy.
2. For each layer $h = 2, \ldots, H$ and iteration $t = 1, \ldots, T$ with $T = \lceil 1/\epsilon \rceil$:
   - Estimate weight function $\hat{w}_h^t$ via an Algorithm-5-style density-ratio objective using sampled datasets (D_1, D_2).
   - Optimize a policy $\pi_{h,t}$ for layer $h-1$ using PSDP-style backward dynamic programming and reward induced by $\hat{w}$.
3. Build cover mixture

$$p_h = \mathrm{Unif}\left(\pi_{h,1} \circ_h \pi_{\mathrm{unif}}, \ldots, \pi_{h,T} \circ_h \pi_{\mathrm{unif}}\right).$$

A.4 Weight Function Estimation (Shared)

The two repositories use the same tabular state-weight model:

- Parameterization: one scalar $w(s)$ per state (including absorbing state).
- Sample size heuristic:

$$n_{\text{weight}} = \left\lceil \frac{40\left(\log|\mathcal{W}| + \log(1/\delta)\right)}{\epsilon_w^2} \right\rceil,$$

  then clipped by `n_weight_cap`.
- Fitting: closed-form update from transition/state counts induced by (D_1, D_2), with objective trend logged as `objective_before` and `objective_after`.

A.5 Policy Optimization Difference

Tabular variant (`codex_state_w`). For each timestep, $Q(s, a)$ is estimated by empirical mean return from rollout statistics, and the policy is extracted by ϵ-greedy smoothing over $\arg\max_a Q(s, a)$.

Neural variant (`codex_state_w_nn`). For each timestep:

- Build a weighted $(s, a) \to Q$ regression dataset from rollout aggregates.
- Encode states with sinusoidal (x, y) features.
- Train a 2-layer MLP (hidden size 64, ReLU, Adam) to predict action values.
- Evaluate the trained network on visited states and export tabular Q values for downstream policy extraction.

A.6 Training and Evaluation Configuration

Common settings in both scripts:

- `seed`=42, `n_actions`=4, `delta`=0.05.
- `epsilon_w`=0.5, `weight_fit_steps`=500, `n_weight_cap`=1024.
- `psdp_samples`=1000, `psdp_sample_workers`=8, `weight_sample_workers`=8.
- `weight_fit_lr`=0.5, `weight_fit_lr_decay`=1.0, `weight_fit_patience`=300.
- `psdp_epsilon_greedy`=0.0, `use_distance_bonus`=false.
- Per-layer rollout evaluation budget: 500 trajectories for policy mixture and 500 for uniform baseline.

A.7 Infrastructure and Job Launch

Experiments are submitted via SLURM with:

- 1 node, 1 task, 8 CPU cores, 32GB RAM.

A.8 Reproducibility Notes

- Both implementations use deterministic seeding through NumPy generators.
- Multiprocessing is used in both weight sampling and PSDP rollout collection.
- The `codex_state_w_nn` variant requires PyTorch for neural Q fitting.